Merge and Split

Loïc Huang | Thomas Aubertier | Pascal Pham

Polytech Sorbonne Université, MAIN3
20 mai 2024

Résumé

L'algorithme Merge and Split pour une coopération équitable dans les réseaux sans fil est appliqué pour étudier les gains résultant de la coopération entre les émetteurs à antenne unique. L'objectif est de trouver une structure de coalition d'émetteurs qui permet aux utilisateurs de maximiser leur utilité tout en tenant compte du coût de la formation de la coalition. Cette structure algorithmique permet aux émetteurs du réseau sans fil de s'auto-organiser et de former un réseau structuré composé de coalitions stables disjointes.

Table des Matières

1 Introduction **3**

2 Modèle **3**
2.1 Principes . . . 3
2.2 Données du système . . . 3
2.3 Partie calculatoire . . . 4
2.3.1 Contrainte de puissance . . . 4
2.3.2 Utilité . . . 5

3 Merge and split **5**
3.1 Algorithme de formation de coalition . . . 5
3.2 Notion de stabilité . . . 6
3.3 Critère d'équité . . . 6
3.3.1 1ère formule : méthode de partage d'équité . . . 6
3.3.2 2ème formule : méthode d'équité proportionnelle . . . 7
3.3.3 Jain index . . . 7

4 Simulation **7**
4.1 Démarche . . . 7
4.2 Mobilité . . . 9
4.2.1 Évolution du bénéfice . . . 9
4.2.2 Évolution du Jain index . . . 11

5 Conclusion **13**
5.1 Critiques du modèle . . . 13
5.2 Compétences acquises . . . 13

6 Diagramme de Gantt **14**

7 Annexes **15**
7.1 Codes . . . 15
7.1.1 Génération d'une grille simple (sans mouvement) . . . 15
7.1.2 Évolution du système pour le mouvement d'un utilisateur . . . 21

1 Introduction

L'étude des réseaux sans fil pour la formation de coalition est une des principales activités de la recherche informatique, cherchant à optimiser et améliorer le concept sur différents aspects. L'étude de la coopération entre récepteurs à antenne unique pour la formation MIMO virtuelle à l'aide de la théorie des jeux est un exemple des études menées qui prouve qu'une grande coalition stable de tous les utilisateurs peut se former si et seulement si aucun coût de coopération n'est pris en compte, ou bien l'étude de la coopération entre émetteurs et récepteurs à antenne unique dans les réseaux ad hoc nous donne des données sur les gains de capacité que les utilisateurs peuvent obtenir en coopérant. Les études se sont limités jusqu'à présent à la formation de grande coalition avec négligence des coûts de coopération. Ce coût n'est plus négligeable lorque la distance séparant les utilisateurs devient trop importante, ainsi l'algorithme Merge and Split va permettre d'apporter une solution à ce problème.

2 Modèle

2.1 Principes

Lorsque les émetteurs à antenne unique coopèrent, ils forment différentes coalitions disjointes et ces coalitions transmettent avec accès multiple par répartition temporelle (TDMA). Ainsi pendant un intervalle de temps, chaque coalition est en mesure d'émettre dans tous les créneaux actuellement occupés par les utilisateurs.

Si une coalition occupe un créneau, une partie de la contrainte de puissance de transmission va être utilisée tandis que l'autre partie va constituer le coût de l'échange d'information entre les membres de la coalition.

Le coût défini dépend de la localisation des utilisateurs et de la taille de la coalition. Le coût de l'énergie est plus élevé lorsque la distance entre les utilisateurs ou la taille de la coalition augmente.

Dans le système TDMA considéré, chaque coalition transmet dans un créneau horaire, sans recevoir d'interférence par les autres coalitions durant la transmission.

L'utilité représente la capacité totale atteinte par la coalition au cours d'un intervalle de temps tout en tenant compte du coût lié à la contrainte de puissance. Si le coût en puissance dans la coalition est plus important que la contrainte, alors la coalition ne peut se former.

À partir de ces données, nous avons un jeu de coalition avec une utilité transférable et nous cherchons par la formation d'une coalition, une structure de coalition qui permettra aux utilisateurs de maximiser leurs utilités en termes de taux avec un coût en termes de puissance.

2.2 Données du système

La simulation peut être altérée via de nombreux paramètres :

- M_t Nombre d'utilisateurs (transmetteurs). Il s'agit du nombre de participants au réseau, pouvant former des coalitions.
- M_r Nombre d'antennes (récepteurs). Définit le nombre de "créneaux" de connexion disponibles pour les coalitions d'utilisateurs.
- R Largeur du terrain. On se limite à une zone carré avec la station au centre.
- σ Variance du bruit. Dans le cadre de la simulation, cette donnée sera fixée et non calculée.

- ν_0 Taux signal sur bruit. Il s'agit du rapport entre les puissances des signaux "utiles" (ceux qui portent l'information) et du bruit (signaux parasites).
- α Exposant d'affaiblissement de propagation. Ce paramètre représente la vitesse à laquelle le signal réduit en puissance. Dans le cadre des réseaux sans-fils, il est compris entre 2 et 6 et dépend de la géométrie de l'environnement.
- κ Constante sur la perte de signal.
- $\tilde{P}$ Contrainte de puissance par slot. Cela correspond à la puissance alouée par slot de connexion. Elle doit compter le coût de connexion et le coût de formation de la coalition de l'utilisateur.
- L Niveau de bruit. Ce paramètre décrit le bruit ambiant de l'environnement. On s'y réfère comme point de référence pour mesurer les variations de bruit commises par le système actif (réseau).

2.3 Partie calculatoire

2.3.1 Contrainte de puissance

On définit une contrainte de puissance d'émission fixe par créneau horaire (par coalition) qui correspond à la puissance disponible pour tous les utilisateurs de la coalition. Si une coalition occupe un slot, une partie de la contrainte de puissance de transmission va être utilisée tandis que l'autre partie va constituer le coût de l'échange d'information entre les membres de la coalition.

On cherche à évaluer le coût nécessaire à la formation de la coalition. Pour cela, on estime le coût d'échange d'un utilisateur i par rapport au plus éloigné de lui $\hat{i}$ au sein de la coalition.

$$\bar{P}_{i,\hat{i}} = \frac{\nu_0.\sigma^2}{h^2_{i,\hat{i}}} \tag{1}$$

avec $h_{i,\hat{i}} = \sqrt{\frac{\kappa}{d^{\alpha}_{i,\hat{i}}}}$ la perte de puissance entre les utilisateurs i et $\hat{i}$.

Ainsi, le coût de puissance total pour la coalition est donné par la somme des coûts d'échanges de l'ensemble des éléments i de S.

$$\hat{P}_S = \sum_{i=1}^{|S|} \bar{P}_{i,\hat{i}} \tag{2}$$

Le coût de l'énergie est alors plus élevé lorsque la distance entre les utilisateurs ou la taille de la coalition augmente. On définit donc une contrainte de puissance réelle par coalition comme différence entre la contrainte de puissance totale et celle déjà utilisée par la coalition. Il reste ainsi la puissance à alouer pour l'échange entre la coalition et la station.

$$P_S = (\tilde{P} - \hat{P}_S)^+ \tag{3}$$

La notation $(.)^+$ désigne que $P_S = 0$ si on obtient une valeur négative (équivaut à $max(P_S, 0)$). Si le coût de la coalition dépasse la contrainte de puissance par slot, la coalition ne peut se former.

2.3.2 Utilité

Dans ce système TDMA, on peut définir une fonction d'utilité pour toute les coalitions. L'expression de l'utilité ainsi que les intermédiaires de calculs sont plus amplement décrits dans un des articles source[3].

On note H_S la matrice de taille $M_r \times M_t$ et de coefficient $h_{i,j} = \sqrt{\frac{\kappa}{d_{i,j}^{\alpha}}}$, puis λ_i valeurs propres de la matrice $H_S^+ H_S$ (avec H_S^+ la matrice conjuguée transposée de H_S). On garde les K valeurs propres positives.

$$D_i = (\mu - \lambda_i^{-1})^+ \tag{4}$$

μ est déterminé grâce à l'égalité

$$\sum_i D_i = P_S \tag{5}$$

On note ensuite C_S la capacité de la coalition telle que

$$C_S = \sum_{i=1}^{K} D_i \left(log_2(\mu\lambda_i)\right) \tag{6}$$

Cette utilité représente la capacité totale atteinte par la coalition au cours d'un échelle de temps tout en tenant compte du coût lié à la contrainte de puissance. Le second cas énonce que si le coût en puissance dans la coalition est plus grande ou égal à la contrainte, la coalition ne peut se former.

$$\nu(S) = \begin{cases} |S|.C_S & \text{si } P_S > 0 \\ 0 & sinon \end{cases} \tag{7}$$

D'autres méthodes pour évaluer la pertinence des coalitions sont proposées par la suite (3.3).

3 Merge and split

3.1 Algorithme de formation de coalition

L'algorithme consiste à réaliser des opérations simples qui permettent de modifier les partitions en suivant la règle de division (Split) et la règle de fusion (Merge). Un groupe de coalitions est fusionné s'il permet d'améliorer l'utilité totale à partir de la fusion tandis qu'une coalition se divise en petites coalitions s'il permet d'améliorer l'utilité totale. Ainsi, pour résumé, avec $\{S_1, ..., S_k\}$, un ensemble de k utilisateurs,

Règle de fusion (Merge) :

$$\sum_{j=1}^{k} \nu(S_j) < \bigcup_{j=1}^{k} \nu(S_j) \tag{8}$$

Règle de division (Split) :

$$\sum_{j=1}^{k} \nu(S_j) > \bigcup_{j=1}^{k} \nu(S_j) \tag{9}$$

La simulation est une succession de 2 phases. Pendant la première phase qui est la phase de formation de la coalition, les utilisateurs forment des coalitions à partir d'itérations de fusion et division arbitraire répétées jusqu'à terminaison. La phase suivante est l'auto-organisation du réseau en coalitions où la transmission avec accès multiple par répartition temporelle (TDMA) prend place avec chaque coalition qui transmet dans le créneau alloué (2.1). Cette phase de transmission peut prendre beaucoup de temps par rapport à la répétition de la phase de formation de coalition, notamment pour un environnement à faible mobilité où les changements dans la structure de coalition sont rares à cause de la mobilité.

3.2 Notion de stabilité

Lors de la simulation, les joueurs peuvent décider de quitter la partition dans laquelle ils se trouvent pour former une nouvelle partition de joueurs. Une partition est de stabilité $\mathbb{D}_{hp}$ si aucun joueur au sein de cette partition souhaite la quitter à travers l'algorithme Merge and Split. Une partition est $\mathbb{D}_c$-stable si aucun de ses joueurs souhaite la quitter, pour tout algorithme employé (non nécessairement l'algorithme Merge and Split).

Toutes les partitions résultant de ce modèle sont de stabilité $\mathbb{D}_{hp}$. Néanmoins, une forme de stabilité plus forte peut être trouvée en utilisant la stabilité $\mathbb{D}_c$ stricte correspondant à l'unique sortie d'une itération arbitraire quelconque de merge and split en une partition qui maximise la somme des utilités de toutes les coalitions. L'existence de cette stabilité dépend de deux conditions nécessaires et suffisantes:

- $\nu(S_1 \bigcup S_2) > \nu(S_1) + \nu(S_2)$
- $\sum_{i=1}^{|T|} \nu(T_i \bigcup G) > \nu(G)$

La première condition est respectée lorsque pour toutes coalitions disjointes d'utilisateurs S_1 et S_2 appartenant à une même partition, l'utilité de la coalition $S_1 \bigcup S_2$, fusion des 2 petites coalitions S_1 et S_2, est strictement supérieure à la somme de l'utilité de chacune ce ces petites coalitions.

Soit G, une coalition formée de joueurs appartenants à $|T|$ différentes partitions. On dit alors que G est T-incompatible avec T, l'ensemble des partitions. La deuxième condition est donc respectée si la somme de l'utilité de chaque coalition de joueurs issus de la coalition G et appartenant à la même partition est strictement supérieure à l'utilité de G.

Lorsqu'une telle partition satisfait ces conditions, l'algorithme proposé converge vers cette partition avec une stabilité stricte $\mathbb{D}_c$ optimale. Si elle n'existe pas, on obtient un réseau de partition à stabilité $\mathbb{D}_{hp}$. Dans un jeu à transmetteur coopérant, les conditions de stabilité $\mathbb{D}_c$ dépendent de la localisation des utilisateurs du réseau en raison des coûts de coopération.

L'algorithme proposé peut être implémenté d'une manière distributive. Comme l'utilisateur peut détecter la puissance du signal des autres utilisateurs, un signal en chaîne peut être appliqué: les utilisateurs distributeurs peuvent échanger quelques informations de chaîne et ensuite performer un algorithme merge and split, minimisant le coût des échanges.

3.3 Critère d'équité

3.3.1 1ère formule : méthode de partage d'équité

La méthode de partage d'équité égale consiste à diviser le bénéfice également entre les utilisateurs. Ceci mène à des partages non équitables lorsqu'un utilisateur possède une meilleure condition par rapport aux autres.

$$z_i = \frac{1}{|S|}\left(\nu(S) - \sum_{j\in S}\nu(\{j\})\right) + \nu(\{i\}) \tag{10}$$

3.3.2 2ème formule : méthode d'équité proportionnelle

La méthode d'équité proportionnelle permet de résoudre l'inégalité de la première méthode et de réaliser ce partage en prenant en compte le poids ainsi que la compatibilité de chaque utilisateur et de répartir le bénéfice en fonction de l'utilité de chacun.

$$z_i = w_i\left(\nu(S) - \sum_{j\in S}\nu(\{j\})\right) + \nu(\{i\}) \tag{11}$$

avec $\sum_{i\in S} w_i = 1$ et $\frac{w_i}{w_j} = \frac{\nu(\{i\})}{\nu(\{j\})}$

3.3.3 Jain index

La notion de Jain index est une mesure d'équité pour déterminer si les utilisateurs ou les applications reçoivent une part équitable des ressources du système. En l'occurence, à partir des données z_i et $\nu(\{i\})$ de chaque utilisateur i, nous pourrons déterminer si le bénéfice au sein d'une coalition a été bien réparti de façon équitable. Cette donnée permet d'évaluer les 2 méthodes ci-dessus (10) (11) dans le calcul de z_i. Plus le "Jain index" d'une coalition notée J(S) se rapproche de 1, plus la méthode choisie pour le critère d'équité est équitable.

$$J(S) = \frac{(\sum_{i\in S}\frac{z_i}{\nu(\{i\})})^2}{|S|\sum_{i\in S}(\frac{z_i}{\nu(\{i\})})^2} \tag{12}$$

4 Simulation

L'objectif est maintenant d'appliquer l'algorithme Merge and Split afin de visualiser comment les partitions de coalition évoluent dans le temps jusqu'à obtenir une stabilité $\mathbb{D}_{hp}$. Pour cela, nous avons réalisé nos modélisations en Python à l'aide de Jupyter Notebook. Par la suite, nous allons interpréter nos résultats et observer si notre démarche aboutit à une partition $\mathbb{D}_c$-stable. A la fin, nous devons retrouver des résultats concordants à la théorie mais également à l'article [1]. Pour plus d'informations, nous avons joint le code Python en annexe.

4.1 Démarche

Dans un premier temps, nous sommes partis du principe que les utilisateurs ne se déplacent pas pour comprendre les différents cas de figure de formation et de division de coalitions. De plus, pour qu'on puisse interpréter avec certitude nos résultats et comparer avec ceux de l'article, nous avons choisi des données du système (2.2) identiques à celles de l'article. Il y avait cependant une exception, en effet, il nous manquait comme donnée σ, c'est-à-dire la variance du bruit. Ainsi, nous avons émis

l'hypothèse que $\sigma = 10^{-2}$, cette valeur a été choisie suite à différents tests de manière à ce qu'elle répond à nos objectifs.

Dans un deuxième temps, à partir des différentes équations de la partie (2.3), nous avons crée des fonctions pour définir l'utilité qui peut s'appliquer à toute coalition et qui est nécessaire pour faire fonctionner notre simulation.

Une fois cela fait, nous avons programmé l'algorithme Merge and Split. Cependant, nos premiers résultats ne donnent pas de résultats satisfaisants, bien qu'on observe une stabilité $\mathbb{D}_{hp}$, les groupes de coalition formés ne semblent pas optimales. En effet, les conditions pour obtenir une stabilité $\mathbb{D}_c$ dépendaient énormément de l'utilité des utilisateurs au sein d'une coalition, de plus, l'utilité d'une coalition S $\nu(S)$ dépend en grande partie de la distance entre ses utilisateurs. Pourtant, nous remarquons que certains utilisateurs appartiennent à une coalition A alors que pourtant ces mêmes utilisateurs semblent plus proches d'une autre coalition B. La cause vient des règles de Merge et de Split (8) (9) qui ne sont pas assez strictes. Dans le cas précédent, la condition de fusion est respectée, peu importe si l'utilisateur appartient à A ou à B.

Pour rendre la phase de fusion plus perfectible, dans le cas où la règle Merge est respectée, nous avons cherché la distance la plus courte entre un utilisateur de la coalition à fusionner et un utilisateur d'une autre de la partition. Dans le cas où cette distance est la plus courte, les 2 coalitions correspondantes fusionnent. Idéalement, il aurait fallu déterminer l'ensemble des distances entre tous les utilisateurs de la partition pour pouvoir obtenir des coalitions $\mathbb{D}_c$-stables. Cette idée ne semble pas pertinente car nous aurions alors obtenu un algorithme de forte complexité. De plus, la condition est suffisante pour obtenir des résultats cohérents, vous pouvez les voir ci-dessous :

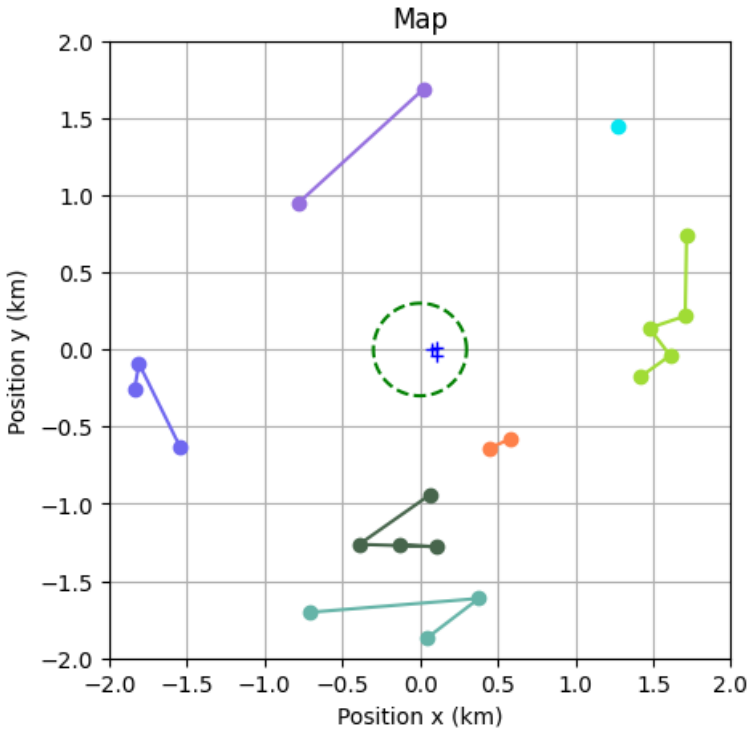

Figure 1: Résultats de l'algorithme sans mobilité avec 20 transmetteurs et 3 récepteurs

Les 3 récepteurs se situent près du centre et les transmetteurs d'une même coalition, qui est représentée par la même couleur, sont reliés entre eux.

4.2 Mobilité

4.2.1 Évolution du bénéfice

Après la mise en place de notre algorithme Merge and Split, nous allons l'appliquer à un environnement dans lequel les transmetteurs peuvent maintenant se déplacer. Afin de pouvoir interpréter correctement nos résultats, nous allons faire déplacer qu'un seul utilisateur durant la simulation. Nous nous attendons à avoir des fusions de coalition lorsque l'utilisateur se rapproche suffisamment d'une coalition et des divisions de coalition dans le cas contraire.

Nous allons simuler le déplacement de l'utilisateur 1 vers les x croissants et allons visualiser l'évolution de l'utilité individuelle de chaque utilisateur en fonction de la distance parcourue par l'utilisateur 1. Seul l'utilisateur 1 verra son utilité variée lorsqu'aucun merge ou split ne se produit car la distance le séparant de la base varie. De plus, pour une coalition, l'utilité individuelle des utilisateurs sera calculé avec la 1ère formule de partage d'équité.

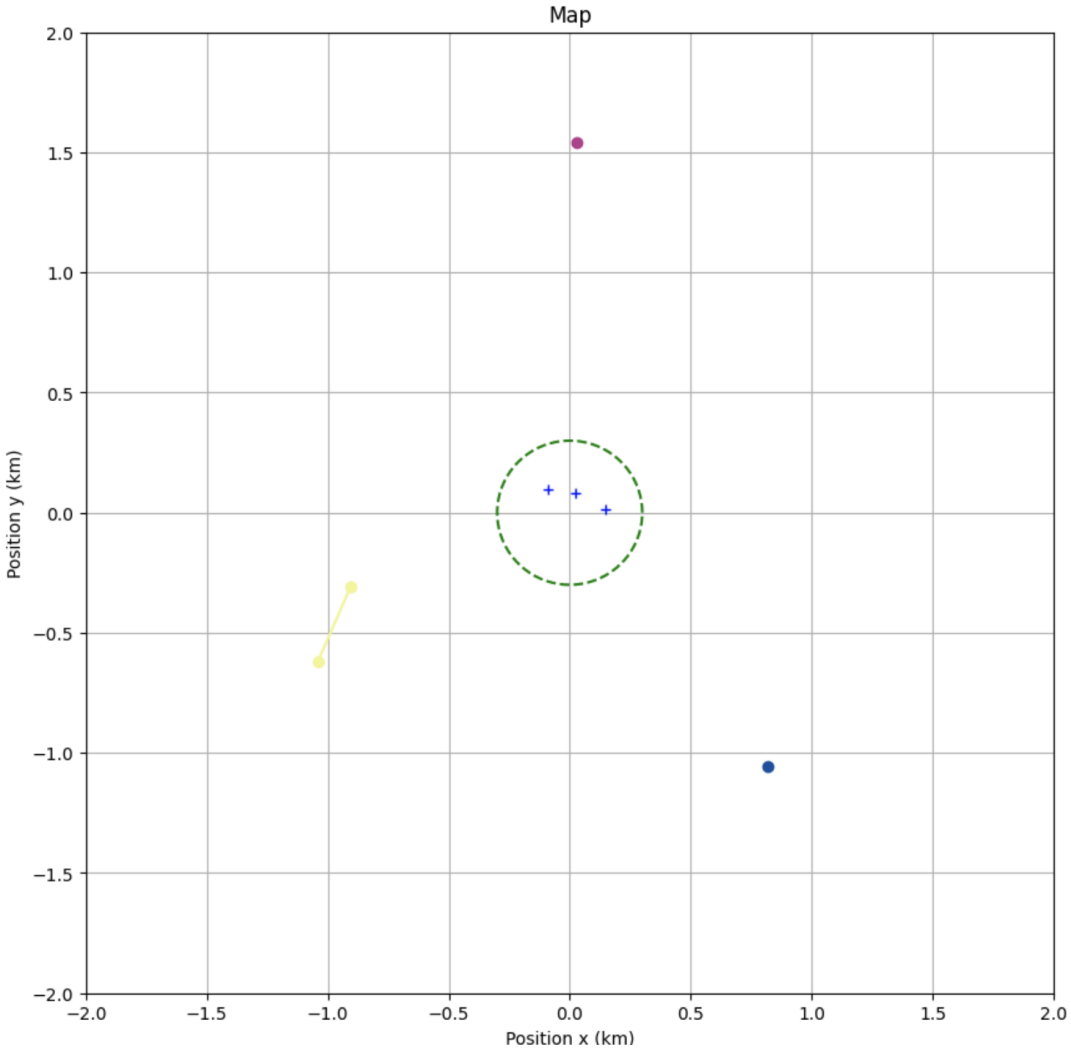

Figure 2: Génération d'une map de 4 utilisateurs

Les coordonnées des utilisateurs 1, 2, 3 et 4 sont respectivement $[-1.04, -0.62]$, $[0.82, -1.06]$, $[0.03, 1.54]$ et $[-0.90, -0.31]$.

Le graphe de l'utilité individuelle des utilisateurs en fonction de la distance parcourue par l'utilisateur 1 est donné ci-dessous:

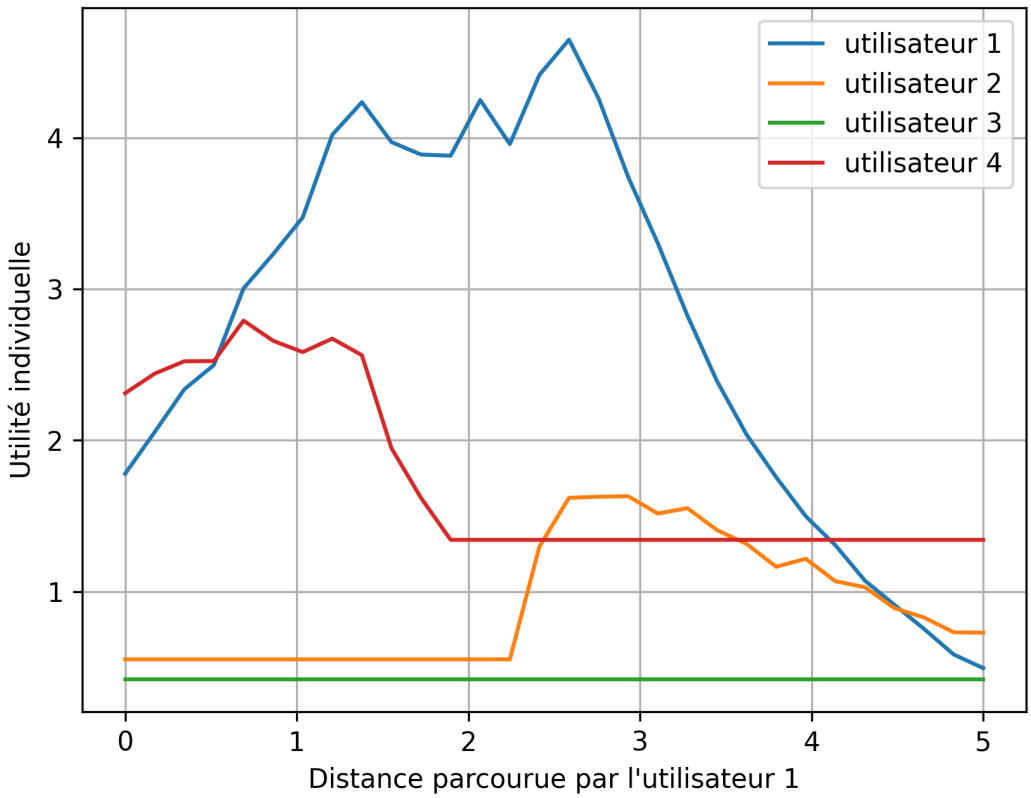

Figure 3: Utilité individuelle des utilisateurs en fonction de la distance parcourue par l'utilisateur 1

L'utilisateur 1 et 4 se trouvent dans une même coalition au début de la simulation et un split se produit aux alentours de 1.9km parcourues par l'utilisateur 1. L'utilisateur 4 retrouve son utilité d'origine et reste constant jusqu'à la fin car il est immobile. Ainsi, on a bien une amélioration de l'utilité individuelle lorsqu'il rejoint une coalition.

Un merge se produit avec l'utilisateur 2 lorsque l'utilisateur 1 a parcouru 2.3km, ce qui est cohérent avec la map. Ce merge a permis à l'utilisateur 2 d'améliorer son utilité mais on observe bien l'inégalité de la 1ère formule de partage d'équité avec la perte d'utilité individuelle de l'utilisateur 1. La 2ème formule faisant intervenir le poids des utilisateurs permettra de corriger ce problème en partageant l'utilité selon la contribution de chaque utilisateur.

Finalement, l'utilisateur 3 qui n'a ni subi de Merge, ni de Split, voit son utilité individuelle constante durant toute la simulation.

L'évolution de l'utilité du point 1 montre bien deux pics étant corrélés à l'apogée de chacune des coalitions auquel il a appartenu, et aucun point ne perd à entrer en coalition (le minimum correspond toujours à l'isolation d'un point). Ces résultats rentrent en accord avec ceux de l'étude.

4.2.2 Évolution du Jain index

Les exemples suivants on été réalisés à partir de 12 utilisateurs, sans modifier d'autres paramètres. Les positions initiales de chaque point sont aléatoires.

Premier exemple :

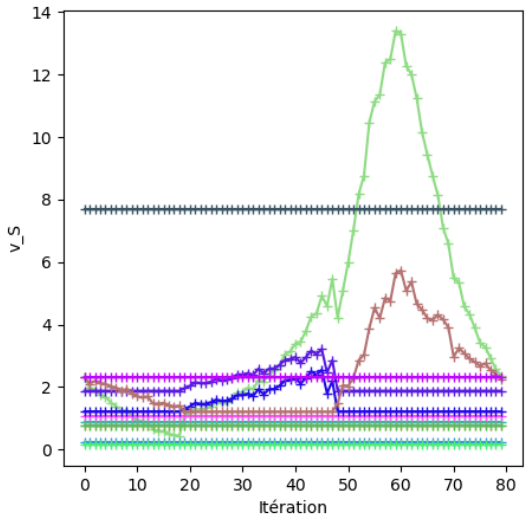

Figure 4: Utilité individuelle des utilisateurs en fonction de la distance parcourue par l'utilisateur 1 (vert clair)

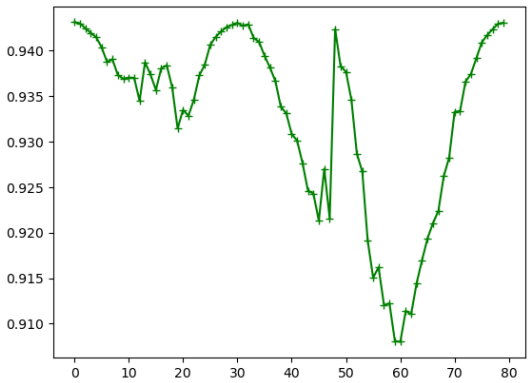

Figure 5: Moyenne du Jain index de l'ensemble des coalitions en fonction de la distance parcourue par l'utilisateur 1

Deuxième exemple :

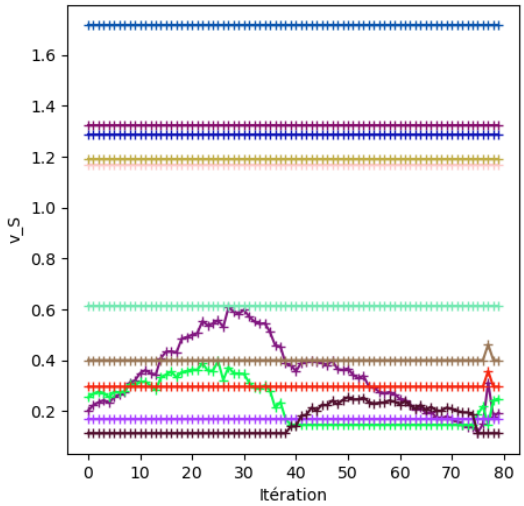

Figure 6: Utilité individuelle des utilisateurs en fonction de la distance parcourue par l'utilisateur 1 (violet foncé)

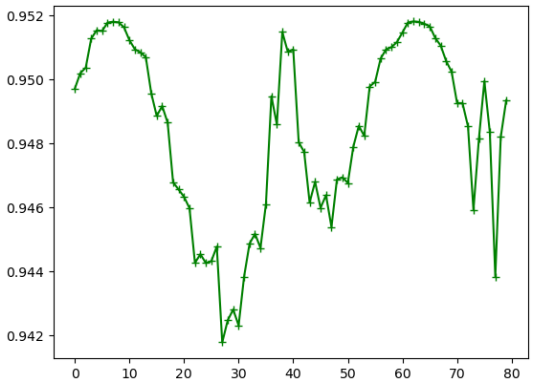

Figure 7: Moyenne du Jain index de l'ensemble des coalitions en fonction de la distance parcourue par l'utilisateur 1

La répartition de l'utilité est réalisée selon la **méthode du partage d'équité** (10). Dans tous les cas, l'index est très élevé, ce qui signifie que cette méthode de répartition est assez équitable entre les membres.

L'évolution de l'utilité individuelle de chaque point suit les mêmes comportements que précédemmant. Cependant, on remarque que le Jain index semble inversement proportionnel au bénéfice total des différents utilisateurs.

En effet, plus l'utilité d'un point augmente, plus l'écart qui le sépare des autres augmente. Malgré un gain global, une meilleur disposition induit souvent une plus grande inégalité entre ceux qui bénéficient de positions avantageuses et ceux qui tirent que peu profit de la coalition.

Par exemple, en observant la figure 4 (4), le grand pic situé vers la soixantième itération correspond à un fort rapprochement entre le point 1 (vert clair) et les antennes au centre. L'écart en lui et le second membre de la coalition (en marron) s'est renforcé, même si cette situation a profité aux deux.

Cependant, si la répartition de l'utilité est réalisée selon la **méthode d'équité proportionnelle** (11), le Jain index est maximisé.

Troisième exemple :

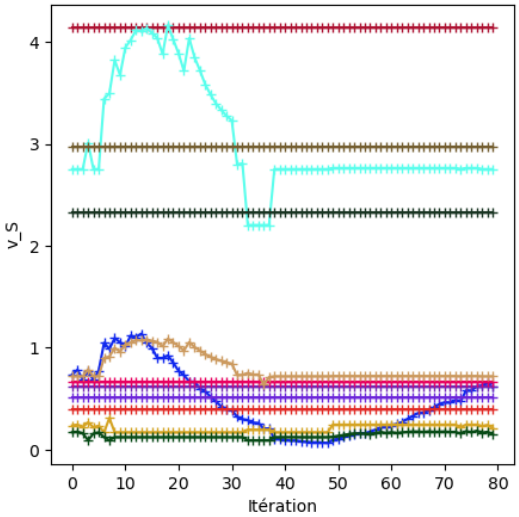

Figure 8: Utilité individuelle des utilisateurs en fonction de la distance parcourue par l'utilisateur 1 (bleu foncé)

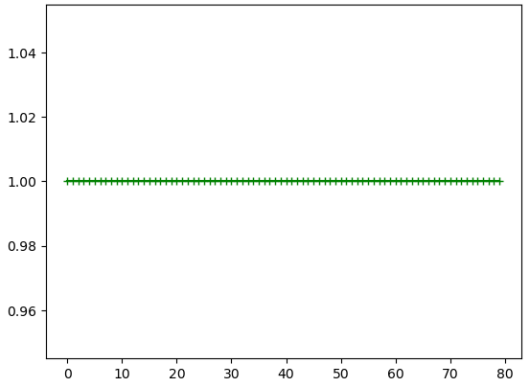

Figure 9: Moyenne du Jain index de l'ensemble des coalitions en fonction de la distance parcourue par l'utilisateur 1

Cette méthode respecte beaucoup mieux l'allocation entre les membres. On remarque par exemple que l'évolution des courbes bleue claire et beige sont proportionnellement identiques pendant la majorité des itérations (les deux points font partie de la même coalition). L'évolution du bleu foncé diffère car celui-ci s'éloigne du centre.

Cette méthode a tendance à donner plus d'utilité aux membres "faibles" de la coalition, quitte à réduire le gain maximum de celui le plus proche des antennes. Cela permet d'assurer un réseau de qualité à un plus grand nombre d'utilisateurs, mais donne plus de poids aux connexions de mauvaise qualité (tous ne sortent pas gagnants de cette répartition).

5 Conclusion

5.1 Critiques du modèle

Le modèle utilisé lors des simulations tente de se rapprocher des contraintes réelles concernant la télécommunication (perte de signal, contrainte de puissance). Cependant, il est évident que beaucoup d'autres paramètres influant sur la qualité de la connexion ne sont pas considérés, comme la variation de la qualité du signal (courant, météo, type d'antenne) ou de la réception de celui-ci (type de forfait/opérateur, obstacles, puissance de l'appareil). Il s'agit d'un modèle "idéaliste", qui ne varie qu'en fonction de la position des utilisateurs.

De plus, la puissance de calcul peut devenir rapidement conséquente. Lors de nos travaux sur le mouvement d'un point, nous avons simulé 80 itérations avec 12 utilisateurs en environ 10 secondes. Or, dans un vrai réseau, le nombre total d'utilisateurs peut largement atteindre plusieurs milliers, voire millions en grande agglomération. En prenant en compte le déplacement permanent de chacun, il devient difficile d'envisager qu'un tel système puisse fonctionner en direct. En revanche, il peut être appliqué au sein de communautés ou municipalités, sur des appareils plutôt statiques (comme des routeurs ou boxs).

Le choix de l'allocation peut également faire débat. Il peut être judicieux d'accorder plus d'importance aux connexions de mauvaises qualités ou éloignées, mais cela vient au détriment des autres utilisateurs. Les questions éthiques et/ou techniques à ce sujet ne peuvent être répondues avec parfaite objectivité.

5.2 Compétences acquises

Ce projet nous a tout d'abord permis de développer nos compétences informatiques. Malgré l'utilisation d'outils connus (LaTeX, Python, Jupyter), la découverte de l'algorithme Merge and Split dans le contexte de points (et non de traitement d'image par exemple) permet d'envisager différemment l'approche de la résolution de problèmes d'optimisation.

Nous avons également pu découvrir la lecture critique d'article scientifiques. Nous avons dû débattre et faire des choix concernant les points difficiles ou peu explicites afin d'avancer, en particulier la partie *"II. System Model"* ([1]). Il y a eu un vrai travail de recherche, et nous avons adapté les décisions de l'articles pour pouvoir obtenir des résultats convaincants.

Nous avons également partagé et expliqué nos différents progrès individuels aux autres membres du groupe, ce qui a encouragé la vulgarisation de notre travail. Nous avons pu améliorer notre compréhension du sujet de cette manière.

Enfin, nous tenons à remercier Mme Francesca Fossati pour nous avoir accordé de précieux rendez-vous et discussions, sans quoi notre simulation n'aurait pas été possible.

6 Diagramme de Gantt

Voici une estimation de la répartition temporelle du travail sur l'ensemble du projet.

TITRE DE LA TÂCHE	PROPRIÉTAIRE DE LA TÂCHE	TÂCHE TERMINÉE (EN %)	PHASE 1					PHASE 2								PHASE FINALE	
			SEMAINE 1	SEMAINE 2	SEMAINE 3	SEMAINE 4	SEMAINE 5	SEMAINE 6	SEMAINE 7	SEMAINE 8	SEMAINE 9	SEMAINE 10	SEMAINE 11	SEMAINE 12	SEMAINE 13	SEMAINE 14	SEMAINE 15
Présentation																	
Découverte du sujet	Tout le monde	100 %															
Contexte	Loïc	100 %															
Réseaux sans fil	Tout le monde	100 %															
Théorie des jeux	Loïc, Pascal	100 %															
Algorithme Merge&Split	Pascal	100 %															
Rapport intermédiaire	Tout le monde	100 %															
Simulation																	
Reproduction de l'exemple	Thomas	100 %															
Mobilité	Loïc, Thomas	100 %															
Scalabilité	Tout le monde	100 %															
Critère d'équité	Pascal	100 %															
Jain index	Thomas	100 %															
Recherche																	
Metaverse	Tout le monde	0 %															
Finalisation																	
Rapport final	Tout le monde	100 %															
Slides	Tout le monde	100 %															

References

[◇] Sources principales :

[1] **Arxiv** (site), A Distributed Merge and Split Algorithm for Fair Cooperation in Wireless Networks, PDF, *Walid Saad*, *Zhu Han*, *Merouane Debbah*, *Are Hjørungnes*, 15 février 2008 7, 13

[◇] Sources complémentaires :

[2] **Roberto Lucchetti** (site), A Primer in Game Theory, *Roberto Lucchetti*, 1 mars 2011

[3] **MIT** (site), Capacity of Multi-antenna Gaussian Channels, *I. Emre Telatar*, décembre 1999 5

7 Annexes

7.1 Codes

Les codes ci-dessous ont été réalisés en Python sur Jupyter Notebook.

7.1.1 Génération d'une grille simple (sans mouvement)

Cette section contient la majorité des fonctions de calcul qui sont utilisées par la suite

```
import numpy as np
import random
from scipy.linalg import hilbert
import matplotlib.pyplot as plt

#Variables globales
landwidth = 2
rmaxdist = 0.2
tmindist = 0.3

Mt = 6
Mr = 3
v0 = 10
alpha = 3
kappa = 1
sigma = 10**(-2)
Pc = 0.01

#Définition fenêtre
fig, biax = plt.subplots()
fig.set_size_inches(5,5)
biax.set_xlabel("Position x (km)")
biax.set_ylabel("Position y (km)")
biax.set_xlim(-landwidth, landwidth)
biax.set_ylim(-landwidth, landwidth)

def CalcDistance(c1, c2):
    return np.sqrt((c2[0]-c1[0])**2 + (c2[1]-c1[1])**2)

def h_ij(c1, c2):
    return np.sqrt(kappa / CalcDistance(c1, c2)**alpha)

def Power_ij(c1, c2, dij):
    hij = np.sqrt(kappa / dij**alpha)
    return v0 * sigma**2 / hij**2

#-- La puissance qui reste à allouer à la connexion (le calcul se fait entre chaque i et î, l'utilisateur de S le plus loin de i)
def Power_S(s):
    if len(s) == 1:
        return Pc
    sum = 0
    for i in s:
        maxd = 0
        for j in s:
            temp = CalcDistance(i, j)
            if temp > maxd:
                maxd = temp
```

```
                    imax = j
            #print("maxd = ", maxd, " et imax = ",imax)
            sum += Power_ij(i, imax, maxd)
            #print("sum = ",sum)
        return max(Pc - sum,0)

def SumDi(mu, lambda_is):
    sumDi = 0
    for lambda_i in lambda_is:
        if lambda_i > 0.001:
            sumDi += max(0, mu - 1/lambda_i)
    #print("sumDi = ",sumDi, " pour mu = ", mu)
    return sumDi

#-- Calcul de mu par force brute
def Calc_mu1(lambda_is, Ps): # Ce que j'ai tenté :
    # Initialisation
    mu = 0.1
    Di = SumDi(mu, lambda_is)
    cpt = 0
    while cpt < 5000: # éviter les boucles infinies (normalement pas nécessaire)
        # Ajout :
        if Di == 0:
            mu += 0.2
        elif (Di - Ps) > 0.001:
            mu -= 0.001
        elif (Di - Ps) < -0.001:
            mu += 0.001

        else:
            return mu
        cpt += 1
        Di = SumDi(mu, lambda_is)
    #print("--------")
    #print("ATTENTION")
    #print("CPT =", cpt)
    #print("--------")
    return mu

#-- Calcul de l'utilité (pour plus de détails, voir l'étude)
def v_s(post):
    #print("arg = ",post)
    Ps = Power_S(post)
    #print("Ps = ",Ps)
    if Ps <= 0:
        return 0

    n = Mr
    m = len(post)
    H = np.zeros((n,m)) #-- Matrice H
    for i in range(n):
        for j in range(m):
            H[i][j] = h_ij(listR[i],post[j])
    #print("H = ",H)
    Ht = np.transpose(H)
    #print("Ht = ",Ht)

    lambda_is = np.linalg.eig(np.dot(Ht,H))[0] #-- Valeurs propres de HtH (les
    lambda_i)
```

```
    for i in range(len(lambda_is)):
        lambda_is[i] = max(0, lambda_is[i])
    lambda_is.sort() #--- Un peu de clarté, ça peut pas faire de mal!
    #print("lambda_is = ",lambda_is)

    #mu = Calc_mu(lambda_is, Ps)
    mu = Calc_mu1(lambda_is, Ps)
    #print("mu = ",mu)

    Cs = 0
    for lambda_i in lambda_is:
        if lambda_i > 0.001: #--- On ne veut que les lambda strictement positifs
            #print(np.log10(mu*lambda_i))
            Cs += max(0,np.log10(mu*lambda_i))
    #print("Cs = ",Cs)

    return 100*m*Cs #--- On fait x100 pour plus de clarté, mais ça ne change rien aux comparaisons

def Linear_Tab(l):
    #print("on passe de ça : ",l)
    taille = len(l)
    i = 0
    while i < taille:
        if type(l[i][0]) is not np.float64:
            for elem in l[i]:
                l.append(elem)
            l.pop(i)
        else:
            i = i+1
    #print("à ça : ",l)
    return l

def powerset(s): #--- Trouvé sur le net puis adaptée
    x = len(s)
    res = []
    for i in range(1, 1 << x):
        res.append([s[j] for j in range(x) if (i & (1 << j))])
    return res[:-1]

def Merge(gliste):
    taille = len(gliste)
    i = 0
    reste = taille-1
    bestmerge = -1
    bestdist = 1000
    bestv = 0
    is_merge = 0
    while reste >= 1:
        temp1 = gliste[reste]
        temp2 = gliste[i]
        temp = Linear_Tab([gliste[reste], gliste[i]])
        #--- On compare le i-ème élément avec le reste-ième (de base, le dernier)
        v = v_s(temp)
        if v_s(temp1) + v_s(temp2) < v: #--- Condition du merge
            dist = CalcDistance(temp1[0], temp2[0])
            if dist < bestdist:
            #if v > bestv:
                bestmerge = i
```

```
                mer = temp
                bestdist = dist
                bestv = v
                #print("Pour ",temp1," : Le meilleur c'est ",temp2," (dist = ",CalcDistance(temp1[0], temp2[0]),"  et  v = ",v,")")
        #--- On compare les 0<i<reste-1 éléments avec le reste-ième avant de décrémenter reste
        i = i+1
        if i >= reste:
            if bestmerge != -1:
                gliste.pop(reste)
                gliste.pop(bestmerge)
                gliste.append(mer)
                bestmerge = -1
                bestdist = 1000
                bestv = 0
                is_merge = 1
                #print(">>>>>>>>>>>>>>>>>>>>> MERGE")
            reste = reste - 1
            i = 0
        #print("-----------------------------------------------------------")
        #print("i = ",i,"  et  reste = ",reste)
        #print(gliste)
    #print("Merge : ",gliste)
    return gliste, is_merge


def Split(gliste):
    taille = len(gliste)
    s = 0
    is_split = 0
    while s < taille: #--- On traîte l'ensemble des subsets initiaux (ceux créés par le split sont ignorés)
        subset = gliste[s]
        #print("subset = ",subset)
        subt = len(subset)
        notsplit = True
        if subt > 1:
            pwrset = powerset(subset) #--- Fonction qui renvoie l'ensemble des sous-ensembles de subset (sauf le vide et lui-même)
            t = len(pwrset)
            i = 0
            while i < t//2 and notsplit:
                if v_s(pwrset[i]) + v_s(pwrset[t-i-1]) > v_s(subset): #--- Condition du split
                    #--- Il se trouve que pwrset[i] et pwrset[t-i-1] sont complémentaires (forment subset à eux deux mais n'ont aucun élément en commun)
                    gliste.pop(s)
                    gliste.append(pwrset[i])
                    gliste.append(pwrset[t-i-1])
                    #print(">>>>>>>>>>>>>>>>>>>>> SPLIT")
                    notsplit = False
                    taille = taille - 1 #--- On a enlevé un subset à traiter, donc on réduit la taille de 1
                    s = s - 1 #--- On contrebalance le +1 qui arrive (l'élément suivant obtient l'indice du précédent)
                    is_split = 1
                i = i+1
        s = s + 1 #--- On passe au subset suivant
```

```
    #print("Split : ",gliste)
    return gliste, is_split

#--- Appel de base, les calculs ne sont pas faits dans cette fonction
def Merge_Split(gliste):
    limit = 0
    while limit < 10: #--- Pour éviter les boucles infinies (on estime que la configuration finale est obtenue avant)
        #MERGE
        #print("###### Tour ",limit," ######")
        #print("------Merge------")
        gliste, mer = Merge(gliste)
        #SPLIT
        #print("------Split------")
        gliste, spl = Split(gliste)
        if mer + spl == 0: #--- Condition de terminaison de l'algorithme (stabilité)
            break
        limit = limit + 1
    return gliste

#--- Génération du terrain
def GenerateGrid():
    #--- Les antennes
    posr = [[0,0] for i in range(Mr)]
    for i in range(Mr):
        posr[i] = 2*rmaxdist* np.random.random_sample(2) - rmaxdist ## [-0.2, 0.2]
        print("Antenne ",i,": ",posr[i]) ## Des antennes pas à l'origine (à voir)

    #--- Les utilisateurs
    post = [[[0,0]] for i in range(Mt)]
    for i in range(Mt):
        #--- Répartition aléatoire...
        while post[i][0][0]**2 + post[i][0][1]**2 < tmindist**2: ## "around the BS" => donc en dehors du cercle : OK
            post[i] = [2*landwidth* np.random.random_sample(2) - landwidth]
            post[i][0] = np.append(post[i][0],[0])
            post[i][0] = np.append(post[i][0],[i])

        #--- ...ou contrôlée
        #post[i] = 0.4*np.random.random_sample(2)
        #if i < 4:
        #    post[i][0] += 0.5
        #    post[i][1] += 0.6
        #else:
        #    post[i][0] += -1
        #    post[i][1] += -0.6
    print("post = ",post)
    return posr, post


#--- Une fois les coalitions créées, on plot chaque point avec la couleur de leur coalition (aléatoire)
def plot_points(gliste, lr, r_color):
    angle = np.linspace( 0 , 2 * np.pi , 150 )
    plt.plot(tmindist * np.cos( angle ), tmindist * np.sin( angle ), '--', color = "green")
    for antenne in lr:
        plt.plot(antenne[0],antenne[1], color = "blue", marker = "+")
    for i in range(len(gliste)):
```

```
        ## Note : on peut donc avoir la même couleur pour 2 coalitions différentes
        plt.plot([j[0] for j in gliste[i]],[j[1] for j in gliste[i]], color = r_color
    [i], marker = "o")

def random_color_list():
    r_color = [tuple(np.random.random_sample(3)) for i in range(Mt)]
    return r_color

def utilite_finale(gliste, l_init):
    sum = 0
    for subset in gliste:
        sum += v_s(subset)
    sum_init = 0
    for pt in l_init:
        sum_init += v_s(pt)
    print("Utilité finale avec coalitions = ",sum," !")
    print("Utilité sans coalitions = ",sum_init," !")

#-- Déroulement du programme
listR, listT = GenerateGrid()
M_S = listT[:]
M_S = Merge_Split(M_S)
print("ET LE GRAND RESULTAT EST : ",M_S)
utilite_finale(M_S, listT)
r_color = random_color_list()
plot_points(M_S, listR, r_color)

plt.title("Map")
plt.grid(True)
plt.show()

```

7.1.2 Évolution du système pour le mouvement d'un utilisateur

```
import matplotlib.animation as animation
from matplotlib.animation import PillowWriter
from operator import itemgetter

#Variables globales
landwidth = 2
rmaxdist = 0.2
tmindist = 0.3

Mt = 12
Mr = 2
v0 = 10
alpha = 3
kappa = 1
sigma = 10**(-2)
Pc = 0.01

nb_frames = 80

#Définition fenêtre
fig, biax = plt.subplots()
fig.set_size_inches(5,5)

def fair(s,v_coal, v_sum, v_indiv):
    #return (1/s)*(v_coal - v_sum) + v_indiv #Méthode 1
    return (v_indiv/v_sum)*(v_coal - v_sum) + v_indiv #Méthode 2

def Jain_index(coal):
    s = len(coal)
    sum1 = 0
    sum2 = 0
    for i in range(s):
        sum1 += coal[i][2]/v_s([coal[i]])
        sum2 += (coal[i][2]/v_s([coal[i]]))**2
    return sum1**2/(s*sum2)

def v_tab(gliste):
    Jain = []
    for coal in gliste:
        v_coal = v_s(coal)
        v_sum = [v_s([pt]) for pt in coal]
        v_sum = sum(v_sum)
        for i in range(len(coal)):
            coal[i][2] = fair(len(coal), v_coal, v_sum, v_s([coal[i]]))
        Jain.append(Jain_index(coal))
    Jain = np.mean(np.array(Jain))
    return gliste, Jain

def MS_update(frames):
    biax.clear()
    biax.set_xlabel("Position x (km)")
    biax.set_ylabel("Position y (km)")
    biax.set_xlim(-landwidth, landwidth)
    biax.set_ylim(-landwidth, landwidth)
    M_S = listT[:]
    M_S[0][0][0] += 0.05
    if M_S[0][0][0] > 2:
```

```
        M_S[0][0][0] = -landwidth
    M_S = Merge_Split(M_S)
    plot_points(M_S, listR, not_r_color)
    M_S, Jain = v_tab(M_S)
    v_list.append([pt[2] for pt in sorted(Linear_Tab(M_S), key=itemgetter(3))])
    Jain_list.append(Jain)
    biax.set(title=frames)
    plt.grid(True)
#-- Déroulement du programme
listR, listT = GenerateGrid()
r_color = random_color_list()
not_r_color = [(0,0.7,0) for i in listT] #Pour plus de lisibilité, on fixe la couleur pour l'animation

v_list = []
Jain_list = []

anim = animation.FuncAnimation(fig, MS_update, frames=nb_frames, interval=2, repeat=False)
anim.save("mobilite.gif", dpi=300, writer=PillowWriter(fps=8))

#-- Utilité
v_list = np.array(v_list)
biax.clear()
biax.set_xlabel("Itération")
biax.set_ylabel(r"v_S")
intervalle = np.arange(0, nb_frames)
for i in range(Mt):
    plt.plot(intervalle,v_list[1:,i], color = r_color[i], marker = "+")
plt.show()
biax.clear()

#-- Jain index
fig.set_size_inches(5,5)
biax.set_xlabel("Itération")
biax.set_ylabel(r"Jain index moyen")
intervalle = np.arange(0, nb_frames)
plt.plot(intervalle,Jain_list[1:], color = (0,0.5,0), marker = "+")
plt.show()


```